

Prototyp „BundleBite“ – Skizze

Beschreibung

„BundleBite“ ist eine kollaborative Plattform für Sammelbestellungen bei Restaurants.

Nutzer können Gruppenbestellungen bei Restaurants erstellen, welche für einen begrenzten Zeitraum (z.B. 5 Minuten) für andere Nutzer in räumlicher Nähe sichtbar bleiben. Andere Nutzer können diesen Gruppenbestellungen beitreten und eigene Gerichte hinzufügen, um gemeinsam Mindestbestellwerte oder kostenlose Lieferungen zu erreichen.

Funktionale Anforderungen

- Nutzer können sich registrieren und anmelden
- Nutzer können offene Sammelbestellungen in ihrer Nähe suchen und filtern (Nach Restaurant, verbleibender Zeit)
- Nutzer können eine neue Sammelbestellung bei einem Restaurant erstellen, mit Standort und Zeitfenster
- Nutzer können ein oder mehrere Gerichte zu einer offenen Sammelbestellung hinzufügen
- Nutzer können ihre hinzugefügten Gerichte in einer laufenden Sammelbestellung bearbeiten oder entfernen
- Nutzer werden benachrichtigt, wenn ein Gericht zu ihrer Sammelbestellung hinzugefügt oder entfernt wird
- Nutzer werden benachrichtigt wenn eine Bestellung abgeschickt oder storniert wird
- Restaurants können sich als eigene Nutzerart anmelden und registrieren
- Restaurants können Gerichte zu ihrer Karte hinzufügen, bearbeiten und entfernen
- Restaurants können Essenskategorien bearbeiten erstellen und löschen
- Gruppenbestellungen werden nach Ablauf eines Zeitfensters automatisch geschlossen
- Das System aggregiert mehrere Nutzerbestellungen innerhalb einer Gruppenbestellung
- Das System protokolliert Ereignisse wie Erstellung, Beitritt und Schließen
- Das System Simuliert einen Essenszubereitungs- und Lieferungsprozess mit ablaufenden Timern

Nicht-funktionale Anforderungen

- Änderungen an Gruppenbestellungen werden innerhalb von maximal 2 Sekunden an Clients übertragen
- Gleichzeitige Änderungen an Gruppenbestellungen sind atomar und thread-safe
- Das System nutzt Multithreading dynamisch auf Mehrkernsystemen
- Mehrere gleichzeitige WebSocket-Verbindungen werden stabil verarbeitet

- Der Server ist innerhalb von 15 Sekunden nach Start bereit, Anfragen entgegenzunehmen
- Server beantwortet korrekte Anfragen zu >99,5% ohne internen Fehler
- Server beantwortet bei <50 REST-Anfragen/Sekunde jede Anfrage in unter 0,3 Sekunden

Verteilung

Nebenläufigkeit:

- Gleichzeitige Verarbeitung mehrerer REST-Anfragen auf separaten Threads
- parallel laufende Timer zur automatischen Schließung von Gruppenbestellungen
- multi-Threading bei Hinzufügen von mehreren Bestellungen zu Gleicher Sammelbestellung

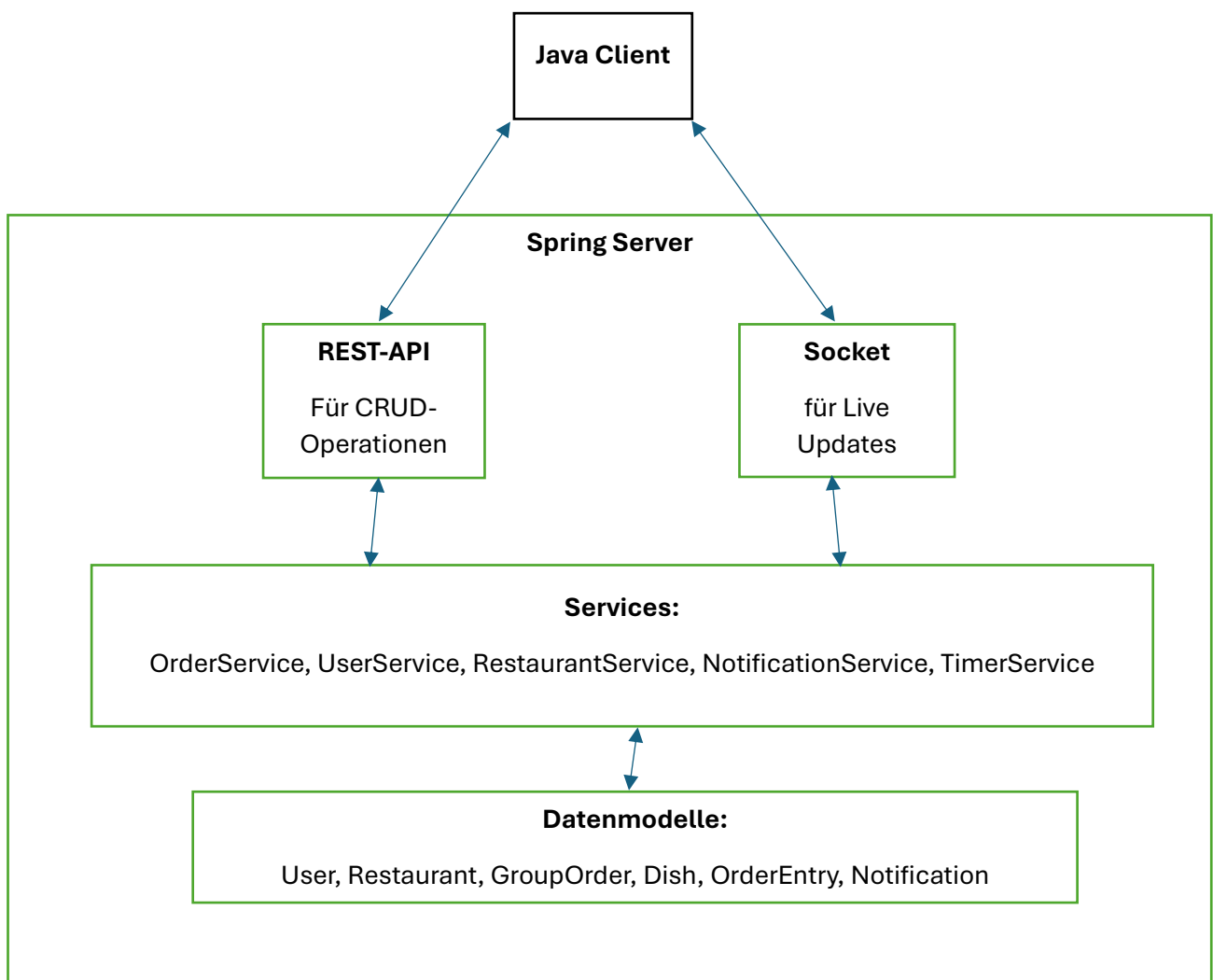
Verteilung:

- Websocket – Benachrichtigungen an Besteller, wenn ein Gericht zur Sammelbestellung hinzugefügt oder entfernt wird
- Websocket – Benachrichtigung an alle Teilnehmer bei Änderungen (Gerichtsänderungen, Abschluss, Stornierung)
- Echtzeit – Update des Bestellfortschrittes für alle Teilnehmer

Komponenten:

- Client-Server Architektur mit Spring Boot
- REST-API für CRUD-Operationen auf Bestellungen
- Services für die Business-Logik und Verwaltung von API-Calls
- In-Memory-Datenhaltung

Diagramm Serverfunktionalität



Lastsimulation und Test der nicht-funktionalen Anforderungen

Nicht-Funktionale Anforderung	Überprüfung
Änderungen an Gruppenbestellungen werden innerhalb von max. 2 Sekunden an Clients übertragen	Ein Client erstellt eine Sammelbestellung, ein zweiter Client fügt der Sammelbestellung eine Speise hinzu. beide Seiten tracken den Zeitstempel. Die Differenz zwischen Änderungszeitpunkt und Eingang der Benachrichtigung wird ausgegeben und gegen den Schwellwert geprüft.

Gleichzeitige Änderungen an Gruppenbestellungen sind atomar und thread-safe	Testclient startet 1000 gleichzeitige Anfragen, ein Gericht zur selben Gruppenbestellung hinzuzufügen. Danach wird der Zustand der Bestellung und der tatsächlich hinzugefügten Gerichte abgefragt.
System nutzt Multithreading dynamisch auf Mehrkernsystemen	Multithreaded Tasks (Timer, parallele REST-Verarbeitung) werden geloggt. Auswertung prüft, ob die Last gleichmäßig auf plausible Anzahl an Threads verteilt ist.
Mehrere gleichzeitige WebSocket-Verbindungen werden stabil verarbeitet	Testclient baut 10 gleichzeitige WebSocket-Verbindungen auf und hält sie offen. Server-Logs werden auf Fehler, Connection-Drops oder Speicherlecks überwacht. Ergebnis: Anzahl stabiler Verbindungen vs. Fehlerrate.
Server startklar innerhalb von 15 Sekunden	Spring Boot loggt Startzeitpunkt und Zeitpunkt der ersten erfolgreich bearbeiteten Anfrage.
Server beantwortet korrekte Anfragen zu >99,5 % ohne internen Fehler	Anfragenergebnisse werden geloggt. Server-Logs unterscheiden zwischen Client-Fehlern (Fehlercode: 4xx) und internen Fehlern (Fehlercode: 5xx). Testclient sendet ein breites Spektrum gültiger Anfragen. das Verhältnis interner Fehler zu Gesamtanfragen wird ausgewertet.
Server beantwortet bei <50 REST-Anfragen/s jede Anfrage in unter 0,3 Sekunden	Testclient sendet Anfragen in definierter Rate mit Taktung. Jede Anfrage wird mit Zeitstempeln versehen. die Antwortzeit wird gemessen und als Erfolg/Misserfolg protokolliert, je nach Antwortzeit. Auswertung: Anteil der Anfragen unter 0,3 s.